

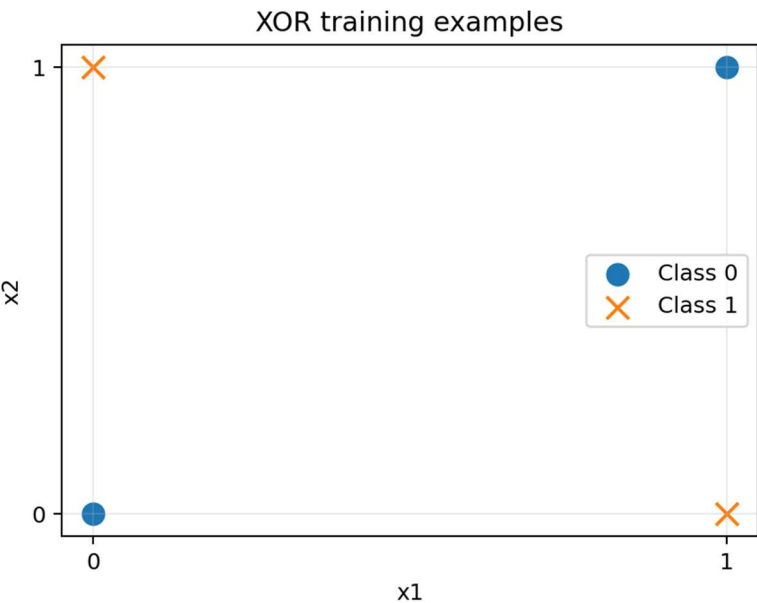
Laboratory Report: Neural Models

Learning, Depth, Activations, and Output Layers — XOR Experiment

1. Task 1 — Problem Specification

The task uses two redundant binary sensor inputs. The device should raise a warning exactly when one sensor is active and the other is inactive, giving the XOR rule.

x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0



The positive and negative examples occupy diagonally opposite corners, so no single straight decision boundary can separate them. A single affine transformation followed by a sigmoid still has a linear decision boundary and therefore cannot represent XOR exactly.

Linear-only baseline (seed 2, Adam, learning rate 0.1, 2000 steps): initial BCEWithLogits loss = 0.699384093; final loss = 0.693147242; probabilities $\approx$ [0.500464, 0.500898, 0.500918, 0.501351]; thresholded predictions = [1, 1, 1, 1].

2. Task 2 — Model Design and Validation Criteria

Architecture: 2 inputs $\rightarrow$ 2 hidden units $\rightarrow$ 1 output. Hidden activation: sigmoid for the main run, followed by tanh and ReLU in the activation experiment. Output: one raw logit, with sigmoid used only when reporting probabilities. Loss: BCEWithLogitsLoss. Optimiser: Adam.

The hidden nonlinearity is scientifically necessary because affine layers compose into another affine map. Nonlinear intermediate features are therefore required for XOR. Sigmoid plus binary cross-entropy is a sensible pairing for a single yes/no target; BCEWithLogitsLoss implements this pairing stably.

Validation criteria: substantial loss reduction; all four correct thresholded labels; nonzero first-layer gradients; reproducibility under a fixed seed; and the expected symmetry behaviour under identical/zero initialisation.

3. Task 3 — LLM-Assisted Implementation

Exact LLM prompt used:

Generate minimal PyTorch code for the following laboratory experiment. Do not change the architecture or task. Use the four XOR examples $[[0,0],[0,1],[1,0],[1,1]]$ with labels $[0,1,1,0]$. Implement a 2-input -> 2-hidden-unit -> 1-output network, with a nonlinear hidden activation, sigmoid binary prediction, random initialisation, and BCEWithLogitsLoss. Use full-batch training for a few thousand CPU steps. Set a random seed for reproducibility. After training, report the initial and final loss, all four probabilities, thresholded labels, and one first-layer gradient tensor after backward(). Keep the code concise and explain where the forward pass, loss, backward pass, and optimizer update occur. Also make it easy to repeat the run with sigmoid, tanh, and ReLU while changing only the hidden activation.

Corrections/engineering changes made before execution:

- Used raw logits with BCEWithLogitsLoss and applied sigmoid only when reporting probabilities.
- Added a fixed seed and explicit first-layer gradient-norm measurement.
- Held the architecture fixed and changed only the activation in the activation comparison.
- Added the required zero-initialisation symmetry experiment and three-class extension.

4. Task 4 — Execution and Results

4.1 Basic learning check

Main sigmoid run (seed 2, Adam, learning rate 0.1, 2000 full-batch steps): initial loss = 0.698453784; final loss = 0.000223114; early first-layer gradient norm = 0.001808615.

Input	Target	Probability	Prediction
(0,0)	0	0.000180343	0
(0,1)	1	0.999764030	1
(1,0)	1	0.999761280	1
(1,1)	0	0.000237337	0

All four XOR labels are correct. The combination of large loss reduction and correct predictions is evidence of useful learning rather than merely a nonzero gradient.

4.2 Backpropagation check

For the first-layer weight matrix, `parameter.grad` represents $\partial L / \partial W(1)$, the sensitivity of the scalar loss to each first-layer weight. For a mean loss over four examples, the total gradient is the average of the four example-wise gradients.

Gradient tensor from the initial backward pass:

$[[0.00046187, 0.00054753], [0.00371669, 0.00392572]]$

Euclidean norm $\|\nabla W(1)L\|_2 = 0.005453269$.

4.3 Symmetry experiment

All parameters were set to zero before training, with the architecture unchanged. The two hidden rows remained identical. In this exact all-zero setup, the first-layer gradients were also zero, so the hidden weights remained zero. Identical hidden units receive identical gradients and cannot spontaneously learn distinct features under deterministic gradient descent.

Step	Loss	Hidden weights	Hidden gradients
0	0.693147	[[0, 0], [0, 0]]	[[0, 0], [0, 0]]
1	0.693147	[[0, 0], [0, 0]]	[[0, 0], [0, 0]]
4	0.693147	[[0, 0], [0, 0]]	[[0, 0], [0, 0]]
9	0.693147	[[0, 0], [0, 0]]	[[0, 0], [0, 0]]
19	0.693147	[[0, 0], [0, 0]]	[[0, 0], [0, 0]]

4.4 Activation experiment

Only the hidden activation changed; seed, architecture, optimiser, learning rate, number of steps, and data were held fixed.

Hidden activation	Final loss	4/4 correct?	Early $\ \nabla W(1)L\ _2$
Sigmoid	0.000223114	Yes	0.001808615
Tanh	0.000081754	Yes	0.039291203
ReLU	0.000037132	Yes	0.107916281

All three activations classified the four examples correctly in this run. The observed early gradient norms differ because the activation derivative participates directly in the backpropagation chain rule. Sigmoid and tanh can have small derivatives in saturated regions; ReLU has derivative 0 on its negative branch and 1 on its positive branch. These are mechanisms explaining this experiment, not a universal ranking of activations.

5. Task 5 — Three-Class Extension

Classes were defined as 0=(0,0), 1=(0,1) or (1,0), and 2=(1,1). The hidden representation remained two-dimensional; the output changed to three logits and the loss to CrossEntropyLoss.

Final output weight shape = (3, 2). Final loss = 0.000022322. Predicted classes = [0, 1, 1, 2].

Input	Target	P(class 0)	P(class 1)	P(class 2)
(0,0)	0	0.999979260	0.000020743	0.000000017
(0,1)	1	0.000006686	0.999982240	0.000011089
(1,0)	1	0.000005963	0.999982360	0.000011637
(1,1)	2	0.000000012	0.000033145	0.999966860

For the second input, the softmax probabilities sum to 1.000000000. Adding 100 to every logit changed the probability vector by at most floating-point roundoff (approximately $1e-7$). This follows because the common factor cancels in softmax. Subtracting the maximum logit before exponentiating therefore improves numerical stability without changing the mathematical probabilities.

For multiclass cross-entropy, the logit gradient is $p - y$. This gradient pushes probability toward the target class and away from the non-target classes.

6. Reflection Questions

1. What did the XOR experiment demonstrate about the difference between depth and nonlinearity?

Affine depth alone remains affine; XOR requires a nonlinear hidden representation.

2. In your successful run, what evidence showed that backpropagation supplied a useful learning signal rather than merely a nonzero gradient?

The gradient was nonzero, the loss fell from about 0.6985 to about 0.000223, and all four labels became correct. The combination of these measurements demonstrates useful learning.

3. Why did identical/zero weight initialisation prevent the two hidden units from learning distinct features?

Identical hidden units receive identical gradients and remain identical. In the all-zero experiment here, the first-layer gradient was exactly zero, so the hidden weights stayed zero.

4. How did changing the hidden activation affect the gradient you observed? Distinguish the scientific explanation from the engineering observation.

The activation derivative enters the backpropagation chain rule. The observed early norms were approximately 0.00181 (sigmoid), 0.03929 (tanh), and 0.10792 (ReLU). These numerical values are observations for this particular experiment; the derivative mechanisms are the scientific explanation.

5. Why must the output layer and loss be selected together according to the task?

The output representation must match the target: one binary logit with binary cross-entropy for the binary case, versus K logits with multiclass cross-entropy for mutually exclusive K-class prediction.

6. Give one example where the LLM improved your engineering productivity and one example where human verification was essential.

The LLM reduced boilerplate coding effort. Human verification was essential for checking the fixed architecture, loss, gradient inspection, required experiments, and measured predictions.

7. Which tests in this laboratory would you keep if the model were scaled up, and which would become too expensive?

I would retain loss/prediction checks, gradient statistics, reproducibility checks where useful, and targeted diagnostics. Exhaustive finite-difference gradient checks would become increasingly expensive as parameter count grows.

7. Conclusion

The experiments connect representation, optimisation, and output design in a small setting that can be inspected directly. The linear baseline could not represent XOR, while the nonlinear 2–2–1 network learned all four labels. The gradient and symmetry experiments exposed backpropagation and initialisation behaviour, the activation comparison showed activation-dependent gradient differences, and the three-class extension demonstrated why the output layer and loss must match the prediction task.